Compiling C with gcc

From `.c` file to running program — on Windows, macOS and Linux

CPE101 Teaching Assistant

CPE101 Computer Programming King Mongkut's University of Technology Thonburi (KMUTT)

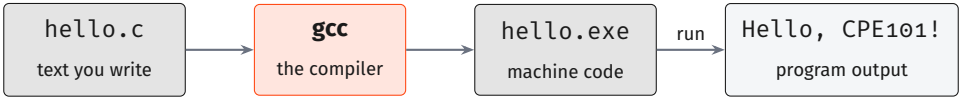
Tutorial session

Today

- What does a compiler do?
- Compiling a single file
- What happens inside `gcc`
- Programs with several files
- Teaser: `make`

The computer cannot read C

You write **source code** (text). The CPU only runs **machine code** (bytes). A **compiler** translates one into the other.



- You compile **once**, then run the executable as many times as you like.
- Change the code ⇒ you must **save and compile again**. The old `.exe` does not update itself.

TIP The compiler "translates" human-readable C code into machine code the CPU can run. Every time you edit the code, you must save and recompile.

Where do I type the commands?

The `gcc` commands are the **same on every OS**. Only the terminal differs.

Windows

Use the **MSYS2 UCRT64** terminal, or PowerShell / the VS Code terminal if `C:\msys64\ucrt64\bin` is on your PATH.

macOS

Use **Terminal**. Here `gcc` is really Apple **clang** under another name. That's fine: every command in this deck still works.

Linux

Use any terminal. `gcc` is the real GNU compiler, usually already installed (package `build-essential`).

Check that it works:

```
$ gcc --version
gcc.exe (Rev5, Built by MSYS2 project) 16.1.0
```

If you see "command not found" / "not recognized", gcc isn't on your PATH. On macOS the first line says `Apple clang version ...` instead.

Just enough terminal to survive

The terminal always works **inside one folder**, the current directory. Your `.c` file has to be in it.

What	PowerShell (Windows)	macOS / Linux / MSYS2
Where am I?	`pwd`	`pwd`
List files	`ls` (or `dir`)	`ls`
Go into a folder	`cd lab01`	`cd lab01`
Go up one folder	`cd ..`	`cd ..`
Run a program here	`.\hello.exe`	`./hello`

Tip: avoid spaces and Thai characters in the folder names you keep code in, e.g. `C:\cpe101\lab01`.

TIP Before compiling, always `cd` into the folder that contains your `.c` file.

Your first compile

```c
#include <stdio.h>

int main(void)
{
    printf("Hello, CPE101!\n");
    return 0;
}
```

hello.c

Simplest possible command:

```
$ gcc hello.c
```

Your first compile

```c
#include <stdio.h>

int main(void)
{
    printf("Hello, CPE101!\n");
    return 0;
}
```

hello.c

Simplest possible command:

```
$ gcc hello.c
```

No message means **success**. gcc only talks when something is wrong.

It made a file with a **default name**:

- Windows: `a.exe`
- macOS / Linux: `a.out`

Your first compile

```c
#include <stdio.h>

int main(void)
{
    printf("Hello, CPE101!\n");
    return 0;
}
```

hello.c

Simplest possible command:

```
$ gcc hello.c
```

No message means **success**. gcc only talks when something is wrong.

It made a file with a **default name**:

- Windows: `a.exe`
- macOS / Linux: `a.out`

Run it:

```
$ ./a.exe      # Windows (PowerShell: .\a.exe)
$ ./a.out      # macOS / Linux
Hello, CPE101!
```

Name your program with -o

`-o name` means “write the **o**utput to this file”.

```
$ gcc hello.c -o hello
```

	File created	How to run
Windows	`hello.exe` (`.exe` is added for you)	`.\hello.exe` (PowerShell), `./hello` (MSYS2)
macOS / Linux	`hello`	`./hello`

TIP Why the `./`? Because macOS/Linux don’t search the current folder for programs automatically. You have to say explicitly “the program right here”.

Name your program with -o

`-o name` means “write the **o**utput to this file”.

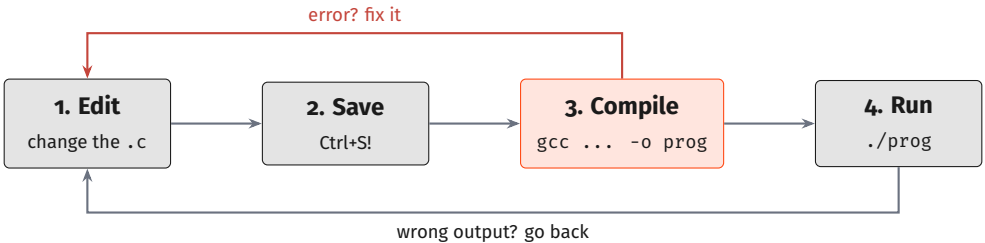
```
$ gcc hello.c -o hello
```

	File created	How to run
Windows	`hello.exe` (`.exe` is added for you)	`.\hello.exe` (PowerShell), `./hello` (MSYS2)
macOS / Linux	`hello`	`./hello`

Common mistake

`gcc hello.c -o hello.c` overwrites your source code with the program! Never put the `.c` name after `-o`.

The edit–compile–run loop



“I fixed it but nothing changed!” The usual reasons:

- The file wasn’t **saved** before compiling.
- You **didn’t recompile**, so you ran the old `.exe`.
- Compiling **failed**, so the old `.exe` is still there. Read the messages!

Turn on warnings: `-Wall -Wextra`

```c
#include <stdio.h>

int main(void)
{
    int score;
    float avg = 7.5;
    printf("avg = %d\n", avg);
    if (score = 10)
        printf("ten!\n");
    return 0;
}
```

Plain `gcc warn.c` compiles this **silently**. But it's buggy.

Turn on warnings: `-Wall -Wextra`

```c
#include <stdio.h>

int main(void)
{
    int score;
    float avg = 7.5;
    printf("avg = %d\n", avg);
    if (score = 10)
        printf("ten!\n");
    return 0;
}
```

Plain `gcc warn.c` compiles this **silently**. But it's buggy.

```
$ gcc -Wall -Wextra warn.c -o warn
warn.c: In function 'main':
warn.c:7:20: warning: format '%d' expects argument of
  type 'int', but argument 2 has type 'double' [-Wformat=]
    7 |     printf("avg = %d\n", avg);
      |                   ~^     ~~~
      |                    |     |
      |                    int   double
      |                   %f
warn.c:8:9: warning: suggest parentheses around
  assignment used as truth value [-Wparentheses]
    8 |     if (score = 10)
      |         ^~~~
```

Two real bugs, found for free. **Always** compile with `-Wall -Wextra`: a warning still produces a program, but it's usually a bug waiting to happen.

Other flags worth knowing

Flag	Meaning
`-o name`	name of the output file
`-Wall -Wextra`	turn on (almost) all the useful warnings
`-std=c11`	use the C11 standard. gcc 15+ defaults to C23, so pick one and use the same as the lab
`-lm`	link the **m**ath library, needed for `sqrt`, `pow`, `sin`, … on Linux. Harmless on Windows/macOS. Put it **at the end**
`-g`	keep debugging info for `gdb` (see the debugging session)

Everyday command, copy this

```
gcc -Wall -Wextra -std=c11 prog.c -o prog -lm
```

Example: a program that uses <math.h>

```c
#include <stdio.h>
#include <math.h>

#define PI 3.14159265358979

int main(void)
{
    double r = 2.0;
    double area = PI * pow(r, 2);
    printf("area = %.2f\n", area);
    printf("sqrt(2) = %.4f\n", sqrt(2.0));
    return 0;
}
```

```
$ gcc -Wall -Wextra -std=c11 circle.c
  -o circle -lm
$ ./circle
area = 12.57
sqrt(2) = 1.4142
```

Example: a program that uses <math.h>

```c
#include <stdio.h>
#include <math.h>

#define PI 3.14159265358979

int main(void)
{
    double r = 2.0;
    double area = PI * pow(r, 2);
    printf("area = %.2f\n", area);
    printf("sqrt(2) = %.4f\n", sqrt(2.0));
    return 0;
}
```

```
$ gcc -Wall -Wextra -std=c11 circle.c
  -o circle -lm
$ ./circle
area = 12.57
sqrt(2) = 1.4142
```

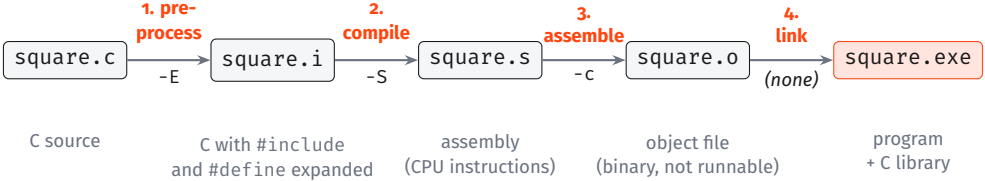
Without `-lm`, on Linux you may get:

```
undefined reference to `pow'
```

That's a **linker** error: the function exists, but its library wasn't linked in. More on the linker soon.

One command, four stages

`gcc` is really a **driver** that runs four tools in a row:



The flag under each arrow means "**stop after this stage**": `-E` prints the preprocessed C, `-S` writes `square.s`, `-c` writes `square.o`. With no flag, gcc goes all the way.

TIP Normally we don't see these stages, because gcc runs all 4 of them in a single command.

Peek at each stage

```c
#include <stdio.h>
#define SIZE 3

int main(void)
{
    int n = SIZE * SIZE;
    printf("%d\n", n);
    return 0;
}
```

square.c

1. Preprocess: `gcc -E square.c` (last lines)

```c
# 4 "square.c"
int main(void)
{
    int n = 3 * 3;
    printf("%d\n", n);
    return 0;
}
```

`SIZE` became `3`. `#include <stdio.h>` was pasted in as *hundreds* of lines above this.

Peek at each stage

```c
#include <stdio.h>
#define SIZE 3

int main(void)
{
    int n = SIZE * SIZE;
    printf("%d\n", n);
    return 0;
}
```

square.c

2. Compile: `gcc -S square.c` → `square.s`

```
main:
        pushq   %rbp
        movq    %rsp, %rbp
        subq    $48, %rsp
        call    __main
        movl    $9, -4(%rbp)
        ...
```

Assembly for an x86-64 CPU. Look: `$9`. The compiler already worked out `3 * 3`.

Peek at each stage

```c
#include <stdio.h>
#define SIZE 3

int main(void)
{
    int n = SIZE * SIZE;
    printf("%d\n", n);
    return 0;
}
```

square.c

3. Assemble: `gcc -c square.c` → `square.o`

A binary **object file**: machine code, but `printf` is still just a *name* that nobody has filled in yet. You can't run it.

4. Link: `gcc square.o -o square`

The **linker** joins your `.o` files with the C library (where `printf` actually lives) into one runnable program.

Want to keep every intermediate file?

```
gcc -save-temps square.c
```

Why should I care? Errors come from different stages

Stage	Typical message	Usual cause
Preprocessor	`fatal error: stdoi.h: No such file ...`	typo in `#include`
Compiler	`error: expected ',' or ';' before ...`	syntax error in your code
Compiler	`warning: unused variable 'x'`	suspicious code (still compiles)
Linker	`undefined reference to 'square'`	function never defined, or a `.c` file / `-lm` missing
Linker	`multiple definition of 'square'`	same function defined in two files

Key idea

`file.c:LINE:COL` ⇒ **compiler**: go to that line. `ld` / `collect2` ⇒ **linker**: check *which files* you gave gcc.

Why split a program into files?

- **Organise**: one file per topic (math, input, display…).
- **Reuse**: use the same functions in many programs.
- **Teamwork**: people edit different files.
- **Faster builds**: recompile only what changed.

TIP Large programs aren't written in one file. They are split into several files by responsibility.

File	Contains
`mathutil.h`	**declarations** (prototypes): *what* exists
`mathutil.c`	**definitions**: *how* it works
`main.c`	`main()`, which uses them

A `.h` file is like a **menu**: it lists the dishes. The `.c` file is the **kitchen** that cooks them.

The three files

mathutil.h

```c
#ifndef MATHUTIL_H
#define MATHUTIL_H

int square(int x);
int cube(int x);

extern int call_count;

#endif
```

mathutil.c

```c
#include "mathutil.h"

int call_count = 0;

/* private to this file */
static int mul(int a, int b)
{
    return a * b;
}

int square(int x)
{
    call_count++;
    return mul(x, x);
}

int cube(int x)
{
    call_count++;
    return mul(x, mul(x, x));
}
```

main.c

```c
#include <stdio.h>
#include "mathutil.h"

int main(void)
{
  printf("square(5) = %d\n",
         square(5));
  printf("cube(3)   = %d\n",
         cube(3));
  printf("calls     = %d\n",
         call_count);
  return 0;
}
```

<stdio.h>: system header (angle brackets)
"mathutil.h": your own header (quotes, looked up in this folder first)

Way 1: compile everything in one command

List **all the `.c` files**. Never list the `.h` files: they get `#included`.

```
$ gcc -Wall -Wextra -std=c11 main.c mathutil.c -o app
$ ./app
square(5) = 25
cube(3)   = 27
calls     = 2
```

Way 1: compile everything in one command

List **all the `.c` files**. Never list the `.h` files: they get `#included`.

```
$ gcc -Wall -Wextra -std=c11 main.c mathutil.c -o app
$ ./app
square(5) = 25
cube(3)   = 27
calls     = 2
```

Forget a file? The compiler is happy, but the **linker** isn't:

```
$ gcc main.c -o app
ld.exe: ...main.c:(.text+0x13): undefined reference to `square'
collect2.exe: error: ld returned 1 exit status
```

Way 1: compile everything in one command

List **all the `.c` files**. Never list the `.h` files: they get `#included`.

```
$ gcc -Wall -Wextra -std=c11 main.c mathutil.c -o app
$ ./app
square(5) = 25
cube(3)   = 27
calls     = 2
```

Forget a file? The compiler is happy, but the **linker** isn't:

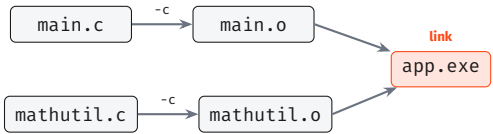
```
$ gcc main.c -o app
ld.exe: ...main.c:(.text+0x13): undefined reference to `square'
collect2.exe: error: ld returned 1 exit status
```

Common mistake

`#include "mathutil.c"`. Never include a `.c` file: pass it to gcc instead.

Way 2: compile separately, then link

```
$ gcc -Wall -Wextra -std=c11 -c main.c
$ gcc -Wall -Wextra -std=c11 -c mathutil.c
$ gcc main.o mathutil.o -o app
$ ./app
square(5) = 25
...
```



`-c` = compile only, **don't link**. Each `.c` becomes a `.o`.

Way 2: compile separately, then link

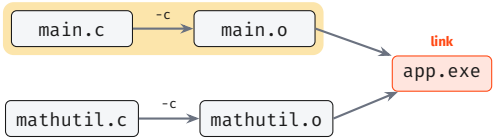
```
$ gcc -Wall -Wextra -std=c11 -c main.c
$ gcc -Wall -Wextra -std=c11 -c mathutil.c
$ gcc main.o mathutil.o -o app
$ ./app
square(5) = 25
...
```

`-c` = compile only, **don't link**. Each `.c` becomes a `.o`.

Changed only `main.c`? Redo just two steps:

```
$ gcc -Wall -Wextra -std=c11 -c main.c
$ gcc main.o mathutil.o -o app
```

In a project with 100 files, this saves a lot of time.



Highlighted: the only part that is rebuilt. (`mathutil.o` is reused as-is.)

Header guards: `#ifndef` … `#endif`

A header can easily get included **twice** (directly, or through another header). Without a guard, its contents are pasted twice:

```
#include "point.h"
#include "point.h"  /* oops */
```

```
$ gcc -std=c11 main.c
point.h:1:8: error: redefinition of 'struct point'
```

TIP A header guard prevents “pasting twice” when the same file is `#included` more than once. Make it a habit to put one in every `.h` file.

Header guards: `#ifndef` … `#endif`

A header can easily get included **twice** (directly, or through another header). Without a guard, its contents are pasted twice:

```
#include "point.h"
#include "point.h"  /* oops */
```

```
$ gcc -std=c11 main.c
point.h:1:8: error: redefinition of 'struct point'
```

Fix: wrap every header like this

```
1  #ifndef POINT_H       /* first time? */
2  #define POINT_H       /* remember it */
3
4  struct point { int x, y; };
5
6  #endif                /* 2nd time: skip
     all */
```

The first include defines `POINT_H`. Any later include sees it's already defined and skips everything.

TIP A header guard prevents "pasting twice" when the same file is `#included` more than once. Make it a habit to put one in every `.h` file.

Sharing between files: `extern` and `static`

Written as	In file	Meaning
`int call_count = 0;`	`mathutil.c`	the variable **really lives here** (definition, outside any function)
`extern int call_count;`	`mathutil.h`	"it exists **somewhere else**": no memory is made here
`int square(int x);`	`mathutil.h`	function prototype: functions are shareable by default
`static int mul(...)`	`mathutil.c`	**private**: other files can't see it

TIP `extern` = declared here, defined elsewhere; `static` (outside a function) = usable only within this file (relates to Question 2 in the mock test).

Sharing between files: `extern` and `static`

Written as	In file	Meaning
`int call_count = 0;`	`mathutil.c`	the variable **really lives here** (definition, outside any function)
`extern int call_count;`	`mathutil.h`	"it exists **somewhere else**": no memory is made here
`int square(int x);`	`mathutil.h`	function prototype: functions are shareable by default
`static int mul(...)`	`mathutil.c`	**private**: other files can't see it

Calling the `static` function `mul` from another file fails at link time (`undefined reference to 'mul'`). That's `static` doing its job.

TIP `extern` = declared here, defined elsewhere; `static` (outside a function) = usable only within this file (relates to Question 2 in the mock test).

Let make type the commands for you

A `Makefile` lists **target: what it needs**, then the command:

```
CC     = gcc
CFLAGS = -Wall -Wextra -std=c11

app: main.o mathutil.o
^^I$(CC) main.o mathutil.o -o app

main.o: main.c mathutil.h
^^I$(CC) $(CFLAGS) -c main.c

mathutil.o: mathutil.c mathutil.h
^^I$(CC) $(CFLAGS) -c mathutil.c

clean:
^^Irm -f *.o app app.exe
```

The indented lines must start with a TAB, not spaces.

```
$ make            # MSYS2: mingw32-make
gcc -Wall -Wextra -std=c11 -c main.c
gcc -Wall -Wextra -std=c11 -c mathutil.c
gcc main.o mathutil.o -o app
```

Let make type the commands for you

A `Makefile` lists **target: what it needs**, then the command:

```
CC     = gcc
CFLAGS = -Wall -Wextra -std=c11

app: main.o mathutil.o
^^I$(CC) main.o mathutil.o -o app

main.o: main.c mathutil.h
^^I$(CC) $(CFLAGS) -c main.c

mathutil.o: mathutil.c mathutil.h
^^I$(CC) $(CFLAGS) -c mathutil.c

clean:
^^Irm -f *.o app app.exe
```

The indented lines must start with a TAB, not spaces.

```
$ make            # MSYS2: mingw32-make
gcc -Wall -Wextra -std=c11 -c main.c
gcc -Wall -Wextra -std=c11 -c mathutil.c
gcc main.o mathutil.o -o app
```

Edit `main.c`, run `make` again:

```
$ make
gcc -Wall -Wextra -std=c11 -c main.c
gcc main.o mathutil.o -o app
```

`make` compares file times and **rebuilds only what changed**: exactly “Way 2”, done for you.

Cheat sheet

I want to...	Command
compile one file	`gcc -Wall -Wextra -std=c11 prog.c -o prog`
...that uses `<math.h>`	add `-lm` at the end
run it	`./prog` (PowerShell: `.\prog.exe`)
compile several files	`gcc main.c util.c -o app`
compile only (no link)	`gcc -c util.c` → `util.o`
link object files	`gcc main.o util.o -o app`
see preprocessed / assembly	`gcc -E prog.c` / `gcc -S prog.c`
prepare for the debugger	add `-g`

Rules of thumb

Always `-Wall -Wextra` • never `-o` a `.c` name • give gcc the `.c` files, never the `.h` • fix the **first** error first

Try it yourself

1. Compile and run `hello.c` with `-o hello`. Then change the message **without** recompiling and run it again. What happens?
2. Compile `warn.c` with and without `-Wall -Wextra`. Fix both warnings.
3. Run `gcc -E square.c > square.i` and open `square.i`. How many lines did `#include <stdio.h>` add?
4. In `multi/`: add a function `int power(int b, int e)` to `mathutil.c` and `mathutil.h`, call it from `main.c`, and rebuild with "Way 2".
5. Remove `mathutil.c` from the gcc command. Read the error. Which stage complained?

All example files are in `01-compiler/examples/`.